

# Cordis `fiber.ts` 源码深度分析

DeepSeek Harness vendored 709 行版本 - 逐函数拆解、生命周期状态机与竞态加固

版本基线: DeepSeek Harness `vendor/cordis/src/fiber.ts`, 检索日期 2026-08-19; 当前文件 709 行。对照: 上游 `cordiverse/cordis` `packages/core/src/fiber.ts` 当前约 458 行。

研究目标: 解释 Fiber 如何把 Cordis 论文中的“可逆 Effect + 响应式 Coeffect”落成可执行的运行时语义, 并重点拆解 `\_runner / epoch / effect / reentrant dispose / reload` 五条主线。

核心判断: `fiber.ts` 不是普通插件生命周期类。它同时承担 ① 插件实例化容器, ② 依赖身份追踪器, ③ Effect ownership 栈, ④ 串行化生命周期协调器, ⑤ 异步清理 join 点。DeepSeek 的 vendored 增强主要针对“生命周期在任何同步回调点都可能重入”的事实。

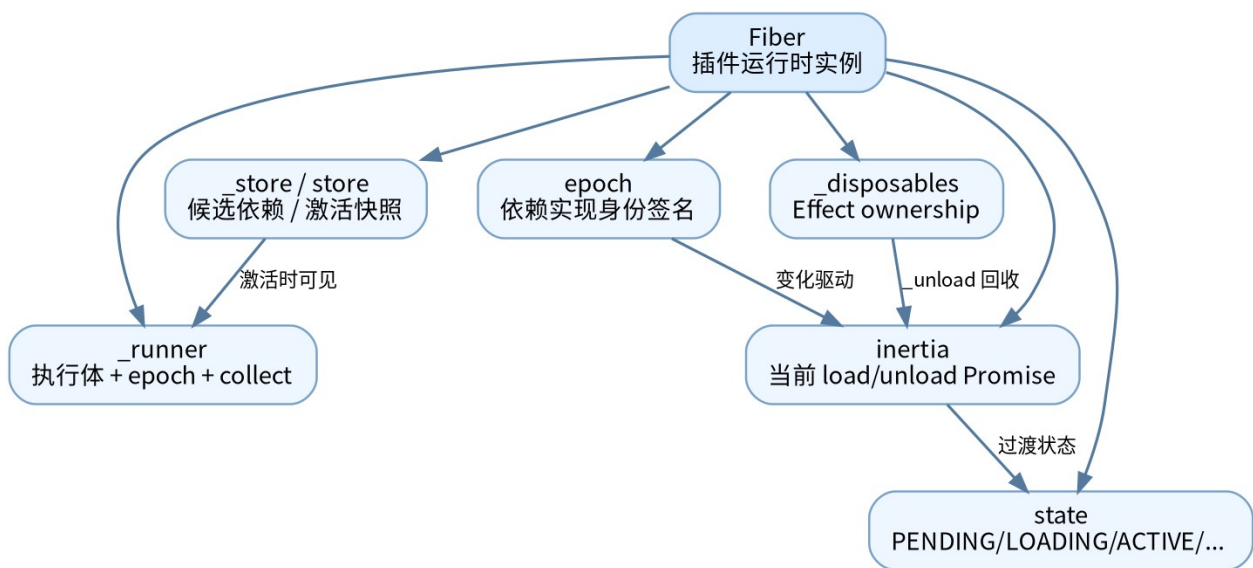


图1 Fiber 内部的六个核心轴: runner、epoch、effect、store、inertia、state

## 0. 阅读结论先行

- `\_runner` 不是普通 callback 封装, 而是“一个可反复激活/失活的 EffectRunner”: `execute` 是启动逻辑, `collect` 把启动过程中产生的 disposer 归属于 Fiber, `epoch` 则是这次激活绑定的依赖版本。
- `epoch` 不是时间戳, 而是依赖 Provider Fiber UID 的有序签名。只要依赖缺失或 Provider 身份改变, epoch 就改变, 从而触发 unload/reload。
- `effect()` 在 DeepSeek 版本里最重要的改动是“先发布 ownership, 再执行用户代码”: wrapper 在 `\_execute()` 之前放入 `\_disposables`, 因此即使 setup 内部同步触发 parent unload, 外层也能看到并等待它。
- `reentrant dispose` 的关键不是“dispose 可以调用两次”, 而是“dispose 可能在 effect 尚未完成 setup、甚至 constructor 正在发布 child 的同步栈上发生”。DeepSeek 增加 setup barrier、inFlight join、WeakMap inertia、PENDING effect drain 来覆盖这些竞态。
- `reload` 采用 epoch-snapshot + microtask checkpoint + stale-epoch guard; `\_setEpoch()` 在已有 `inertia` 时只更新目标 epoch, 不并发启动第二条生命周期链, 最终由当前 load/unload 在尾部观察最新 epoch 并继续切换。

- `_unload()` 在 Fiber 层面对多个顶层 effect wrapper 使用 `Promise.all` 并行回收；但每一个 effect 自己内部的 disposer 仍按逆序串行回收。不要误以为整个 Fiber 的所有资源严格全局 LIFO。

## 1. 源码基线与 709 行的“增量”到底在哪里

当前 DeepSeek Harness vendored 文件为 709 行；上游 Cordis main 的同名文件约 458 行。差异不仅来自 JSDoc。真正影响语义的增强集中在 Effect teardown、constructor publication、config activation 以及 stale lifecycle 四类。

| 主题                 | 上游简化行为                               | DeepSeek Harness 709 行版本的加固                                                        |
|--------------------|--------------------------------------|------------------------------------------------------------------------------------|
| Effect 可见性         | 执行 setup 后才把 wrapper 放入 owner        | 先把 wrapper 放入 owner，再执行用户代码，允许重入 unload 捕获                                         |
| 重复/并发清理            | wrapper 单次标志                         | 增加 <code>disposing/disposalTask</code> ，所有调用者 join 同一清理                            |
| 外层 join 内层清理       | 无专门桥接                                | <code>effectInertia: WeakMap</code> + <code>runDisposable()</code> 连接已开始 cleanup   |
| setup 期间 dispose   | 依赖 <code>task.then(dispose)</code>   | 显式 <code>setupBarrier</code> + <code>disposeAfter()</code> ，区分 executing 与已返回 task |
| 同步 setup 失败        | catch 后本地 dispose                    | 考虑 owner 已抓取 wrapper 的竞态，维护 <code>setupFailed/inFlight</code>                      |
| PENDING Fiber 的副作用 | 通常依赖 epoch 变化触发 unload               | dispose 时若无 inertia，强制 <code>_unload()</code> ，处理 observer 在激活前注册 effect           |
| child 发布顺序         | 较早 emit/plugin                       | 先由 parent 持有 child disposer，再发布 <code>internal/plugin</code>                       |
| reload 过期启动        | 进入 <code>_reload</code> 后执行 callback | microtask 后核对 oldEpoch，失效则不运行插件代码                                                  |
| config             | 构造/更新时直接 resolve                     | 保存 <code>_config</code> 原值，激活时经 <code>internal/config</code> waterfall + schema 解析 |

这组差异共同解决一个问题：Cordis 的 Event、Effect、Plugin publication 都允许同步用户代码，因此生命周期操作不是“未来某个 tick 才发生”，而可能在当前调用栈里反向重入。

## 2. 文件结构：从 709 行切成 20 个逻辑单元

| 源码区间    | 单元                                                 | 职责                                     |
|---------|----------------------------------------------------|----------------------------------------|
| 16-57   | ValidationError / resolveConfig                    | 配置 schema 校验与归一化                       |
| 58-100  | Disposable / Effect / EffectRunner                 | Effect 的返回形态与 runner 协议                |
| 101-127 | effectInertia / runDisposable / emitPluginDisposed | DeepSeek 的 teardown join 与 observer 隔离 |
| 136-163 | FiberState / CordisError / INACTIVE                | 状态、错误与 inactive sentinel               |
| 170-312 | Fiber constructor                                  | 上下文、runner、child ownership、发布与首次依赖解析   |
| 314-330 | name / assertActive                                | 诊断名称与 disposed 防线                      |
| 332-376 | _execute                                           | 统一执行 sync/async/generator effect       |
| 390-527 | effect                                             | Effect 生命周期核心，DeepSeek 加固最密集           |
| 533-537 | getEffects                                         | Effect 元数据树诊断                          |

|         |                          |                                      |
|---------|--------------------------|--------------------------------------|
|         |                          |                                      |
| 539-558 | _getState / _updateState | 派生稳定状态 + provider 状态广播               |
| 559-571 | _checkImpl               | 解析单个 coeffect provider               |
| 572-584 | _refresh                 | 根据 Provider UID 计算 epoch             |
| 585-599 | _setEpoch                | 把 epoch 变化串行化成 load/unload           |
| 600-603 | _resolveConfig           | 激活期 config waterfall + schema        |
| 604-631 | _reload                  | 激活、stale guard、错误转 FAILED            |
| 632-653 | _unload                  | 并行执行顶层 effect cleanup，决定下一跳          |
| 660-665 | await                    | 等待生命周期链完全稳定                          |
| 673-678 | restart                  | 强制 inactive→refresh，形成 unload→reload |
| 690-706 | update                   | 活跃/非活跃两种配置更新策略                       |

### 3. 数据结构先拆：Fiber 里每个字段为什么存在

| 字段           | 语义                                                 | 常见误解                                         |
|--------------|----------------------------------------------------|----------------------------------------------|
| uid          | Registry 内 Provider identity; `null` 表示永久 disposed | 不是对象地址，也不是状态枚举                               |
| ctx/context  | 属于该 Fiber 的 child Context                          | 不是单纯配置对象；它决定 service scope 与 event 视图        |
| config       | 当前激活使用的已解析配置                                       | 不一定等于用户最近提交的 raw config                      |
| _config      | DeepSeek 增加的 raw config                            | 允许在依赖未就绪时延迟解析                                |
| inject       | 声明的 coeffect/service 依赖图边                          | 不仅控制属性访问，还控制生命周期                             |
| _store       | 最新探测到的依赖 Impl 候选                                   | 可在 Fiber 未 ACTIVE 时变化                        |
| store        | 当前激活快照                                             | 插件执行期间使用的稳定依赖视图                              |
| _runner      | 插件级 EffectRunner                                   | 其 epoch 是依赖版本，不是 effect() 内局部 boolean epoch  |
| _disposables | 该 Fiber 拥有的顶层 effects                              | 不仅是用户 disposer，还包括 child plugin wrapper      |
| inertia      | 当前 load/unload 过渡 Promise                          | 起到 lifecycle mutex / serialization token 的作用 |
| _error       | 最近启动/配置错误                                          | 使稳定状态派生为 FAILED                              |

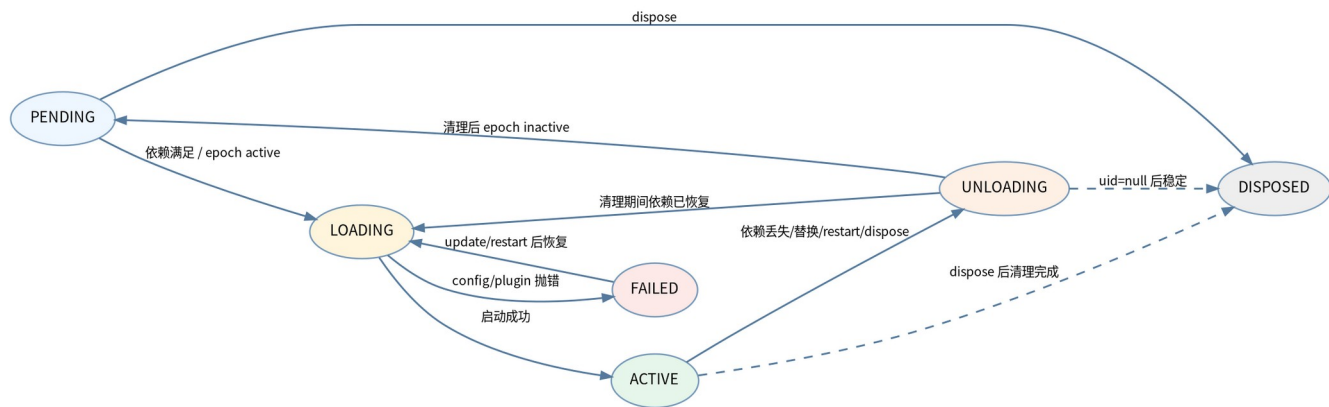


图2 FiberState 的主要迁移；稳定状态与过渡状态分开

## 4. 类型与辅助函数逐个拆解

### 4.1 `ValidationError`：为什么做聚合错误

它把 standard-schema 的多个 issue 合并为一个 TypeError 子类，并给 prototype 挂 `Symbol.for("ValidationError")` 标记。这个 symbol 标记允许跨包/跨 realm 的错误识别比 `instanceof` 更稳。它只负责“配置不合法”的分类，不承担生命周期恢复。

### 4.2 `resolveConfig(runtime, config)`：同步 schema 边界

如果 runtime 没有 Config schema，直接返回原配置；否则调用 Standard Schema 的 validate。当前明确拒绝异步验证，因此 Fiber 激活路径可以把 config resolution 看成同步步骤。DeepSeek 版本在 `\_resolveConfig()` 外再加一层 `internal/config` waterfall，使插件系统可以在 schema 校验前动态改写配置。

重要：DeepSeek `update()` 在 Fiber 非 ACTIVE 时不会立即解析 config，因为 config hook 可能访问 injected service。它只保存 `\_config`，把真正解析推迟到 `\_reload()`。这让“配置解析”本身也服从 coeffect 生命周期。

### 4.3 `Disposable / Effect / AsyncDisposable`：一个 API 接受五种返回形态

`\_execute()` 统一接受：单个 disposer、空值、Promise<disposer>、同步 iterable<disposer>、异步 iterable<disposer>。这允许普通函数、async setup、generator setup 共用同一 Effect 语义。`AsyncDisposable` 既可被调用又实现 PromiseLike，是 Cordis 为“等待 setup 后获得 disposer”设计的双态 API。

| 返回形态                      | 含义                  | _execute 行为              |
|---------------------------|---------------------|--------------------------|
| `() => void/Promise`      | 一个 cleanup          | 立即 collect               |
| `null/undefined`          | 无 cleanup           | 直接结束                     |
| `Promise<disposer>`       | 异步 setup 后给 cleanup | then(safeCollect)        |
| `Iterable<disposer>`      | 逐项注册多个 cleanup      | 同步迭代并 collect            |
| `AsyncIterable<disposer>` | 长期/异步产生 cleanup     | 每次 next 前检查 runner epoch |

## 4.4 `effectInertia` 与 `runDisposable()`：为什么需要 WeakMap

DeepSeek 增加 `WeakMap<Disposable, () => void | Promise<void>>`。公开 disposer 保持 single-shot：第二次调用可以返回空；但 Fiber owner 在 unload 时不能因此“看不到正在进行中的 cleanup”。所以 `runDisposable(d)` 先调用 `d()`，再查询 WeakMap：如果这个 wrapper 已经有 `inFlight` cleanup，就返回它，让外层 join。

伪代码：

```
runDisposable(d):  
  direct = d()  
  return effectInertia[d]?.() ?? direct
```

这个设计把“API 幂等”与“所有权等待语义”分离：对外重复 dispose 不重复执行；对内 owner 始终能够等待已经启动的 teardown。

## 4.5 `emitPluginDisposed()`：为什么不用普通 `emit`

child dispose 时需要通知 `internal/plugin` 观察者，但任何一个观察者的同步 throw 或异步 rejection 都不应该阻断结构性 ownership cleanup。因此函数先安全获取 callback 列表，再逐个执行，每个 callback 都单独 try/catch，并对 Promise rejection 单独记录。

这里体现“结构性 cleanup 优先级高于观察者正确性”：observer 可以失败，但不能让 child 从 parent/runtime registry 中泄漏。

## 5. `Fiber` constructor : 709 行版本最容易被低估的竞态入口

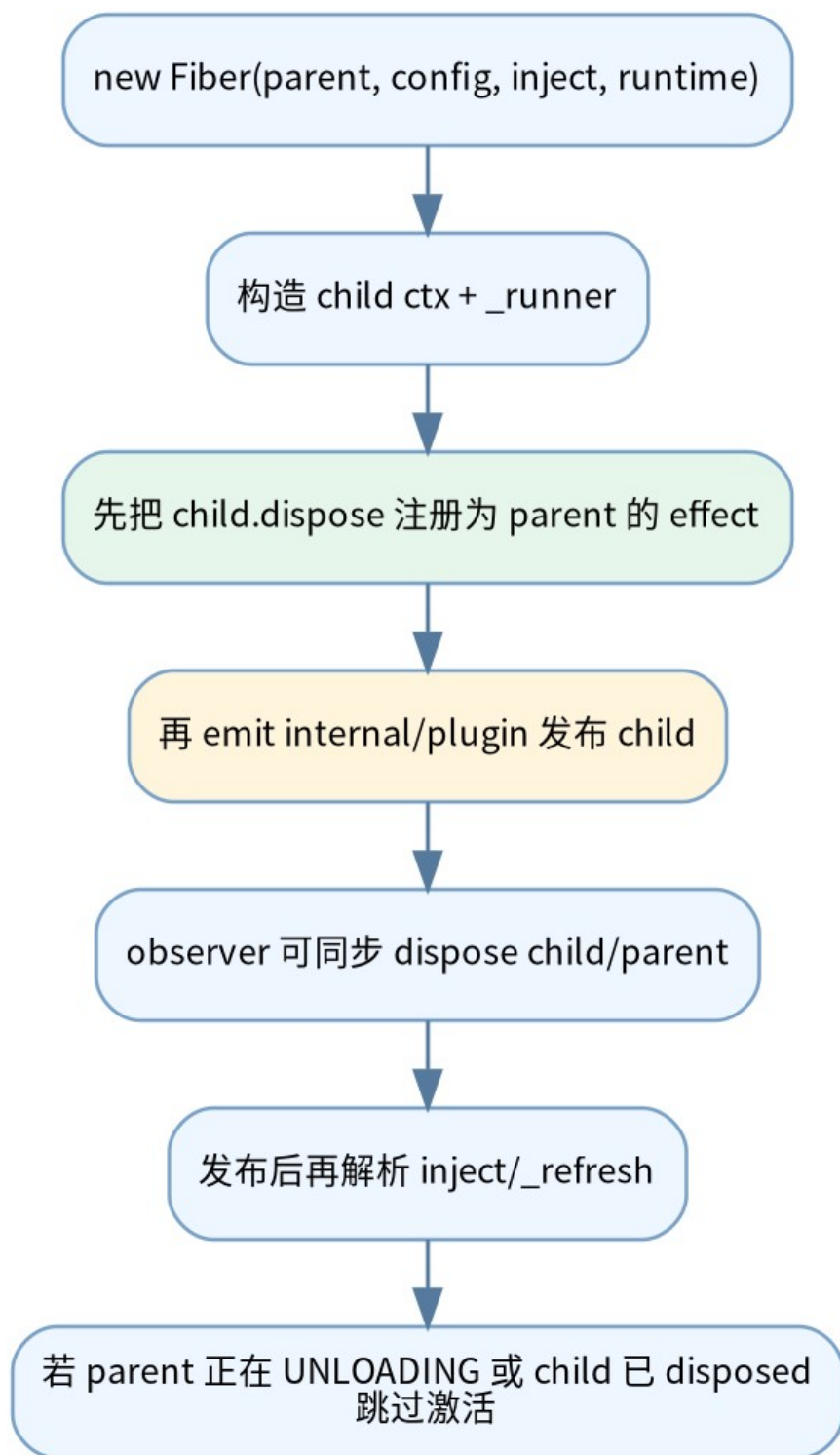


图3 DeepSeek constructor 的关键顺序: 先 ownership, 再 publication, 再 dependency resolution

## 5.1 根 Fiber 与普通 Fiber 是两套语义

``runtime === null`` 表示 root Fiber: `uid=0`、state 直接 ACTIVE、store 为空对象、runner epoch 是空字符串，dispose 实际映射到 restart。普通 Fiber 初始 PENDING，runner epoch=INACTIVE，只有依赖满足后才会进入 LOADING。

## 5.2 `\_config` 与 `config` 分离

构造第一步保存 ``_config = raw config``。`config` 只有在真正激活时才通过 ``_resolveConfig()`` 得到。这样当插件缺依赖时，不会在 PENDING 阶段执行可能依赖 service 的配置 hook。

## 5.3 `parent.extend({fiber:this})`：Fiber 与 Context 的绑定

child Context 的 ``fiber`` 指回新 Fiber。后续 ``ctx.effect()``、``ctx.provide()`` 等 API 都通过当前 Context 的 fiber 把 side effect 归属到正确 owner。inject 上的非空配置还会写入 ``Context.intercept`` 原型链，实现依赖服务的 per-consumer 配置覆盖。

## 5.4 `\_runner` 初始化：插件 callback 被标准化为启动 Effect

runner.execute 同时支持 class plugin 与 function plugin。class 情况会 ``new runtime.callback(ctx, config)``，执行 init hooks，再调用符号化 init；function 情况直接 `callback(ctx, config)`。无论哪种，返回值都被 ``_execute()`` 当作 Effect 结果收集。

## 5.5 最关键的顺序：先让 parent 拥有 child.dispose

DeepSeek 版本先调用 ``parent.fiber.effect(...)`` 创建 child 的结构性 disposer。只有当这个 disposer 已经完整赋给 ``this.dispose`` 后，才 emit ``internal/plugin``。原因是 observer 是同步可执行用户代码：它可能在 publication 回调中直接 dispose child，甚至 dispose parent。若 ownership 尚未建立，就会出现“对象已发布但没有结构性回收路径”的窗口。

## 5.6 publication 失败为什么必须立即自清理

``internal/plugin`` 同步观察者如果抛错，constructor 捕获后会异步调用 ``this.dispose()``，确保 child 从 parent effect、runtime.fibers 和 registry 中退出，然后把原错误重新抛给调用者。这里遵循异常安全里的 strong-ish guarantee：publication 不成功就尽量恢复到“child 未存在”的结构状态。

## 5.7 为什么发布之后才 `\_checkImpl + \_refresh`

历史语义希望 observer 第一次看到 child 时还是 PENDING；更重要的是 loader 可以在 ``internal/plugin`` 回调里扩展 inject。于是依赖解析必须晚于 publication。与此同时 DeepSeek 检查 ``uid !== null`` 且 parent 不在 UNLOADING，避免同步 reentrant disposal 后又继续激活已死 child。

# 6. `\_runner` 深挖：它是 Fiber 的“可重放启动事务描述”

``EffectRunner<T>`` 只有四个字段，但它是 Fiber 的抽象中心：

| 字段      | 插件级 runner                           | effect() 局部 runner                       |
|---------|--------------------------------------|------------------------------------------|
| epoch   | 字符串依赖签名或 `INACTIVE`                  | boolean：该局部 effect 是否仍有效                 |
| execute | 插件 callback / class init             | 用户传给 <code>`ctx.effect()`</code> 的 setup |
| collect | 放入 Fiber <code>`_disposables`</code> | 放入 effect 局部 disposer 列表                 |



|               |              |               |
|---------------|--------------|---------------|
| getOuterStack | plugin 调用外层栈 | effect 创建点外层栈 |
|---------------|--------------|---------------|

## 6.1 为什么同一个 `\_execute()` 能服务两种 runner

因为 Fiber 把“启动插件”和“执行一个局部 effect”都抽象成：执行某段代码，然后收集其逆操作。不同之处仅在 ownership target 与 epoch 类型。插件 runner 的 collect 把资源归到 Fiber；局部 effect runner 的 collect 把资源归到 wrapper 的局部列表。

## 6.2 `\_execute()` 的 `oldEpoch` 快照

对 async iterable effect，循环每次拿下一个 disposer 前都比较 `runner.epoch !== oldEpoch`。如果 lifecycle 已经失效，就停止继续消费 generator。这不是完整取消 generator 的协议，但它阻止旧 activation 在 dependency identity 变化后继续向新世界追加 cleanup。

## 6.3 `composeError()` 的角色

Fiber 自身不手工拼接长 stack，而是把执行包进 `composeError`，并在同步/异步 generator 的关键点强制捕获 Error 作为 async stack 锚点。目标是把“插件创建位置”与“异步 teardown 失败位置”组合起来，改善诊断。

## 7. `\_execute()` 逐分支解释

1. 快照 `oldEpoch`。这代表本次执行绑定的 runner 世代。
2. 构造 `safeCollect`，只接受 function 或 nullish；其它值直接 TypeError，防止错误的 Effect shape 悄悄进入 cleanup 栈。
3. 调用 `runner.execute.call(this)`；此处 `this` 是 Fiber，因此 runner.execute 中的 `this.ctx/this.config` 有意义。
4. 返回 function：当成单 disposer。
5. 返回 null/undefined：认为 setup 无需 cleanup。
6. 返回非 object 且非 function：非法。
7. 返回 PromiseLike：等 Promise 完成后收集 disposer。
8. 返回同步 iterable：不断 next() 并逐个收集；同步异常交给 composeError。
9. 返回 async iterable：先进入微任务以形成异步栈；每轮先检查 epoch 是否仍等于 oldEpoch，再 await next()。

设计意义：`\_execute` 把 Cordis 的“effect 函数”从一个具体 TypeScript 返回类型提升为协议：只要最终能产生 0..N 个 disposer，就能成为可追踪 Effect。



## 8. `effect()` 逐行逻辑：整个文件技术含量最高的 130+ 行

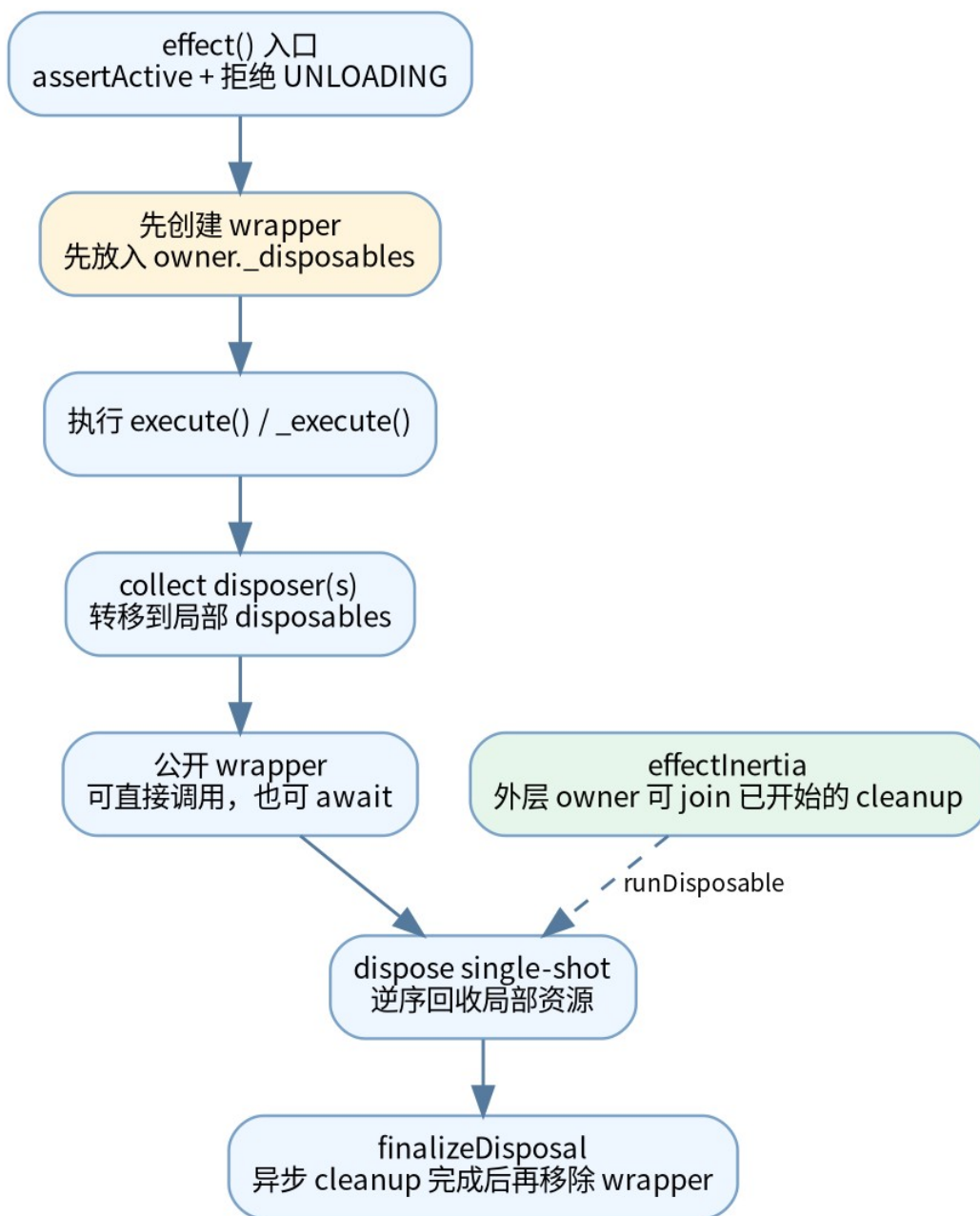


图4 `effect()` 的 ownership 与 teardown 管线

### 8.1 入口防线：disposed 与 UNLOADING 是两个不同条件

`assertActive()` 只检查 `uid !== null`，意味着 PENDING、LOADING、FAILED 仍可能合法注册内部 effect；但 DeepSeek 额外禁止 `state===UNLOADING` 时创建新 effect。原因是 unload 已经对 `\_disposables` 做了快照/清空，如果此时继续注册，会形成无法由本轮 teardown 捕获的新资源。

## 8.2 局部 `disposables[]`：Effect 内部的逆序作用域

每个 `effect()` 都有自己的局部 disposer 数组。`dispose()` 首次调用后设置 `disposing=true`，之后所有调用都返回同一个 `disposalTask`。局部资源按 reverse 顺序执行；一旦出现异步 disposer，后续 disposer 串在其 Promise 之后，保持局部严格 LIFO。

### 一个重要细节：为什么调用 `runDisposable` 而不是直接 `disposable()`

因为数组里的元素也可能是另一个嵌套 effect wrapper。那个 wrapper 的公开调用可能已经被别处触发而返回空，但 WeakMap 里仍记录 inFlight cleanup。`runDisposable` 能拿到正在进行的 Promise 并 join。

## 8.3 EffectMeta：为什么 cleanup 还要构造树

每个 wrapper 绑定 `{label, children}` 元数据。局部 collect 如果发现被收集 disposer 也有 `symbols.effect`，就把其 metadata 作为 child。`getEffects()` 因而能输出 ownership tree，而不仅是一串匿名函数。这个诊断结构对寻找“谁还占着资源”非常重要。

## 8.4 关键变量组：DeepSeek 为 reentrancy 引入的小状态机

| 变量                       | 作用                                                      |
|--------------------------|---------------------------------------------------------|
| executing                | `_execute()` 还没有同步返回；说明 setup 仍处在当前调用栈                  |
| setupBarrier             | 仅当 setup 期间发生 dispose 时才懒创建，用于让 teardown 等 setup settle |
| resolveSetup/rejectSetup | 把 `_execute` 的最终 task 结果桥接给 barrier                     |
| setupFailed              | 同步 setup 已抛错，帮助 owner 侧 wrapper 返回内部 inFlight rollback  |
| inFlight                 | 当前异步 cleanup Promise；供 WeakMap/owner join               |
| removeWrapper            | 把 wrapper 从 owner `_disposables` 中移除的函数                 |

## 8.5 `disposeAfter(setup)`：为什么失败也要先 cleanup 再重新抛错

如果 setup Promise resolve，随后 dispose；如果 setup reject，则先 `await dispose()`，然后重新 throw 原 setup 错误。这样调用者得到的是原始失败原因，同时框架仍保证已经产生的部分资源被回收。它类似 exception-safe constructor：构造到一半失败，也要析构已构造字段。

## 8.6 `finalizeDisposal()`：wrapper 什么时候可以从 owner 消失

若 cleanup 同步结束，立即 removeWrapper；若返回 Promise，则把它包装成 pending，并在 finally 才 removeWrapper，同时清掉匹配的 inFlight。关键不变量是：只要 teardown 尚未 settle，owner 仍应能通过 wrapper 看见并 join 这次清理。

## 8.7 最关键改动：wrapper 在 `\_execute()` 之前发布给 owner

DeepSeek 版本先 `\_disposables.push(wrapper)`，再 `\_execute(runner)`。这一个顺序改变，直接关闭了最危险的 reentrant leak window：用户 setup 内部可以同步触发 parent unload，而 parent 此时已经看得到该 effect。

如果反过来——先 execute，再 push wrapper——那么 setup 里触发的 unload 会清空 owner 当前已知 effects，却漏掉正在 setup 的这一项；setup 返回后 wrapper 才被加入一个已经卸载完成的 Fiber，形成孤儿资源。

## 8.8 wrapper 在 `executing=true` 时被调用会怎样

它不会立刻 dispose，因为 setup 可能还没把所有 cleanup 交出来。它创建/等待 `setupBarrier`，等 `\_execute()` 的 task settle 后再执行 dispose。这样 reentrant unload 不会与 setup 并发撕裂同一个 effect 的资源初始化。

## 8.9 同步 setup throw：最难边界路径

catch 分支把 `executing=false`、`setupFailed=true`、`runner.epoch=false`，然后通过 `finalizeDisposal(dispose)` 回滚已收集资源，并 reject 可能已经存在的 setupBarrier。若 owner unload 已在当前调用栈更早捕获 wrapper，它调用 wrapper 后会通过 `setupFailed ? inFlight : undefined` 等待这次内部 rollback。最后原 setup error 继续抛出。

## 8.10 步 task reject：为什么不制造 unhandled rejection

task rejection 触发 cleanup；如果 effect 已被 wrapper 标记为无效，就直接 dispose，否则走 finalizeDisposal。随后再 catch cleanup 自身错误并记录 logger。这里的目标是“不让 setup rejection 变成未处理 Promise，同时不吞掉 teardown 故障的可观测性”。

## 8.11 `wrapper.then`：这个奇怪 API 的意义

Effect disposer 同时是 PromiseLike。`await ctx.effect(asyncSetup)` 的语义不是“立刻卸载”，而是等待 setup 完成后得到一个可执行的 disposeAsync。实现上先 await task，再映射为 disposeAsync，然后交给 onFulfilled/onRejected。它让同步 effect 与异步 effect API 尽可能统一。

## 9. Reentrant Dispose 深挖：DeepSeek 加固的真正主题

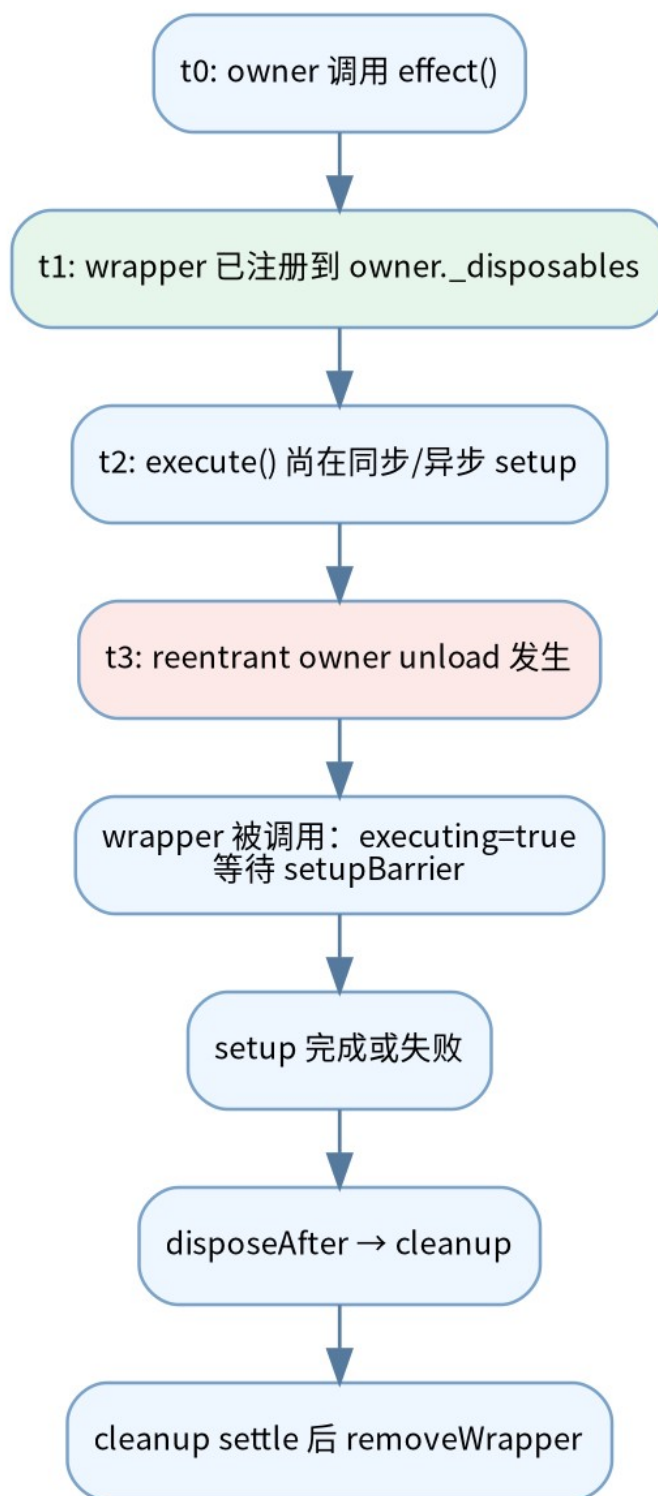


图5 setup 尚未完成时 owner unload 的 reentrant 时间线

## 9.1 什么叫 reentrant，不是“并发”那么简单

JavaScript 单线程也会发生 reentrancy：框架调用用户 callback；用户 callback 同步触发框架的另一个入口；第二个入口反过来修改第一个入口正在建立的数据结构。它发生在同一个调用栈，不需要多线程。

## 9.2 竞态 A：effect setup → parent.dispose()

setup 中触发 parent unload。如果 wrapper 尚未进入 parent `_disposables``，parent 的 snapshot 不包含它。DeepSeek 用“先 push wrapper 后 execute”解决。

## 9.3 竞态 B：wrapper 已被 owner snapshot，但 setup 同步抛错

owner 可能已拿到 wrapper 并准备调用；与此同时 setup throw，公开 wrapper 永远不会返回给插件代码。``setupFailed`` 与 ``inFlight`` 让内部 owner 调用仍能等待 rollback，而不会把失败 effect 当成“已经无事可做”。

## 9.4 竞态 C：用户先 dispose，随后 owner unload

公开 disposer 已 single-shot，第二次直接调用会返回空；但 owner 必须等待第一次异步 cleanup。``effectInertia`` WeakMap 把 wrapper 映射到 ``inFlight`` getter，``runDisposable`` 因而能 join 第一次 teardown。

## 9.5 竞态 D：PENDING Fiber 已经产生 effect

constructor 的 ``internal/plugin`` observer 可以在 Fiber 仍 PENDING 时注册 effect。此时 ``_runner.epoch`` 仍 INACTIVE。如果直接 dispose，``_setEpoch(INACTIVE)`` 发现新旧相同，不会启动 unload。DeepSeek 显式检查 ``!inertia``，强制进入 ``_unload()``，清掉“激活前副作用”。

## 9.6 竞态 E：child publication 时 parent 被同步 unload

DeepSeek 先让 parent 拥有 child disposer，再 emit child publication；emit 返回后又检查 parent state 是否 UNLOADING。如果已在回调中进入 teardown，则 child 不再继续依赖解析与激活。

| 竞态                                 | 如果不修复的后果                               | DeepSeek 防线                           |
|------------------------------------|----------------------------------------|---------------------------------------|
| setup 中 parent unload              | 新 effect 漏出 owner snapshot             | wrapper 提前 publish                    |
| 同步 setup throw + owner 已抓 wrapper  | owner 无法等待 rollback                    | setupFailed + inFlight                |
| 公开 dispose 与 owner unload 重叠       | owner 误以为 cleanup 已结束                  | WeakMap effectInertia + runDisposable |
| PENDING observer effect            | epoch 无变化导致 cleanup 不启动                | dispose 强制 <code>`_unload()`</code>   |
| child publication 重入 parent unload | 已死 child 继续 <code>`_refresh`</code> 激活 | ownership-first + post-emit guard     |

## 10. `epoch` 深挖：Cordis Reactive Coeffect 的最小运行时编码

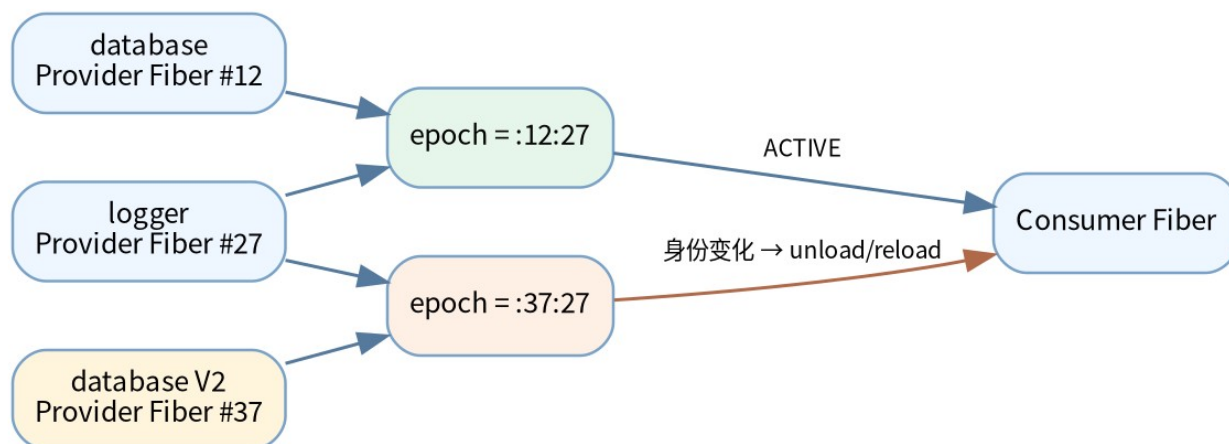


图 6 epoch 不是时间：它编码“当前依赖由哪些 Provider Fiber 提供”

### 10.1 `\_checkImpl(name)`：把 service lookup 变成依赖候选缓存

函数向 ReflectService 请求当前 strict ACTIVE provider；没有则从 `\_store` 删除。有 provider 时还执行 provider 自定义 `check`，且 `check` 以 traceable service value 为 this。check 抛错只记录在 provider logger，并把该依赖当作不可用。

### 10.2 `\_refresh()`：epoch 字符串为什么拼 UID

遍历 inject 的 key。任何依赖缺失，epoch 直接设为 `INACTIVE`；全部存在则按 inject 顺序拼 `:" + impl.fiber.uid`。因此同名 service 从 Provider #12 换成 #37，即使能力名称不变，epoch 也变化。

### 10.3 provider identity 变化为什么一定先 unload

Consumer 的旧 callback 可能已经捕获旧 service reference、注册针对旧 provider 的监听器、生成旧 provider 相关的子资源。只“热换 ctx.database 指针”不足以恢复一致性。Cordis 用 epoch 变化触发完整 unload/reload，把旧 activation 的 effects 回滚后重新执行 callback。

### 10.4 epoch 的局限：字符串签名不是依赖内容 hash

它只追踪 Provider Fiber identity，不追踪 service 对象内部可变状态。Provider 不更换 Fiber 但内部状态变化，不会触发 Consumer reload；那类变化应通过事件、service API 或显式 restart 表达。

## 11. `\_setEpoch()`：一个 15 行左右的 lifecycle serializer

逻辑看起来很短，却建立了核心并发模型：

伪代码：

```
old = runner.epoch
if new == old: return
runner.epoch = new
if inertia exists: return # 只记录目标，不开第二条链
if old inactive && new active:
```

```

    inertia = reload(); state = LOADING
else:
    inertia = unload(); state = UNLOADING

```

关键点是先更新 `\_runner.epoch`，再看 `inertia`。因此当前正在 load/unload 时的新变化不会丢失，只是不立即并发处理。正在执行的生命周期在尾部会比较自己的 oldEpoch 与当前 epoch，决定是否继续下一跳。

这可以理解成 “last desired epoch wins + single in-flight transition”。它不是队列化每一次变化，而是压缩中间状态，只保证最终收敛到最新依赖拓扑。

## 12. Reload 深挖：为什么 `\_reload()` 先 `await Promise.resolve()`

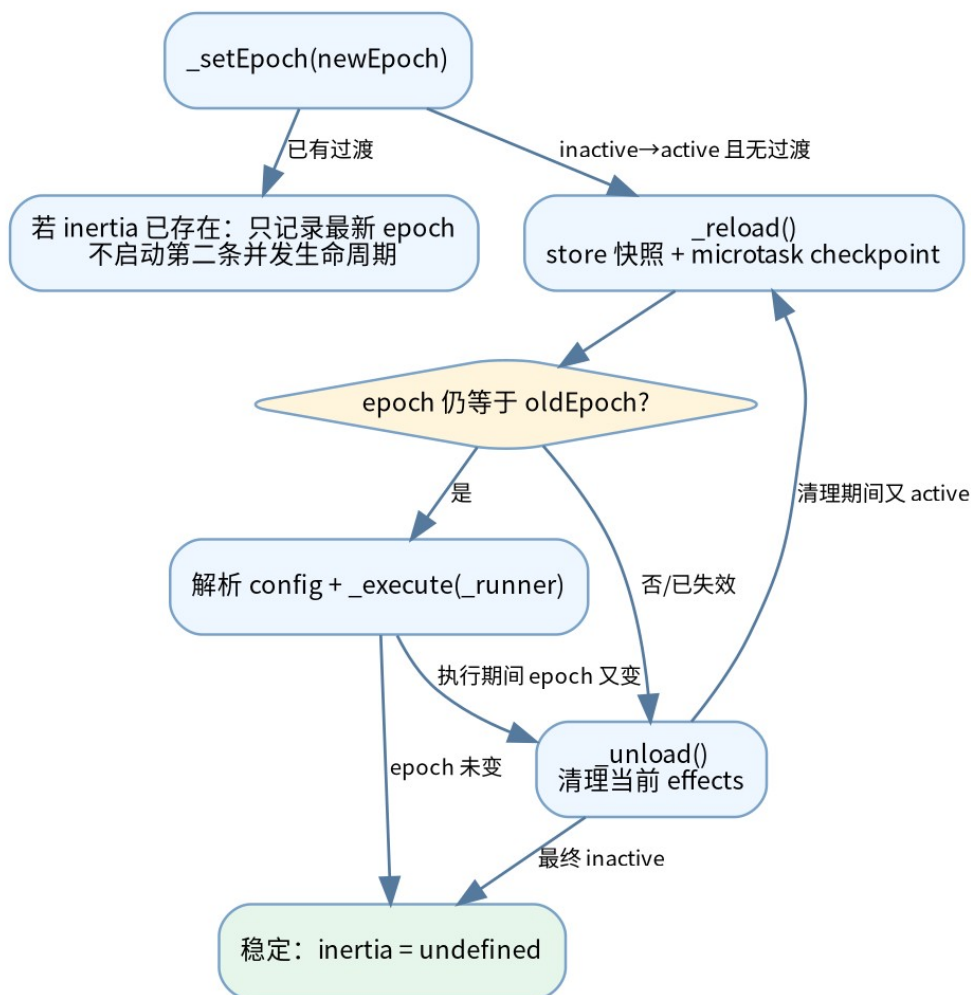


图 7 reload/unload 形成单链式 reconciliation loop

### 12.1 第一步 `store = {...\_store}`：冻结激活视图

`\_store` 是实时依赖候选，可能被 notify 更新；`store` 是当前 activation snapshot。Context service lookup 在 Fiber ACTIVE 时可以基于这个快照保持一次启动期的一致性。



## 12.2 `oldEpoch` 快照：这次 load 属于哪个依赖世代

snapshot 后进入微任务 checkpoint。此时排在当前同步栈后的 dispose/notify 有机会先运行并改 epoch。DeepSeek 再检查 `this.\_runner.epoch === oldEpoch`；只有仍然有效才解析 config 和执行插件。

## 12.3 stale epoch guard 的工程价值

如果刚决定“依赖满足，开始 load”，但在 callback 真正运行前依赖已经被 dispose，那么执行插件只是制造注定要马上清理的新副作用。stale guard 避免这次无意义甚至危险的启动。

## 12.4 config 为什么放在 stale check 之后

因为 config waterfall 本身可能访问 injected services；只有确认当前 epoch 仍有效，才应该解析 config。若 config/plugin 抛错，记录 `\_error` 并把 runner epoch 设回 INACTIVE，后续状态派生为 FAILED/PENDING 语义。

## 12.5 `\_reload()` 尾部为什么可能立刻 `\_unload()`

插件 callback 运行期间可能触发 service graph 变化。若结束时当前 epoch 已不同于 oldEpoch，说明本次 activation 已过时：不能把 inertia 置空，而是直接接 `\_unload()` 并把状态设 UNLOADING。

# 13. `\_unload()` 深挖：两层清理顺序

## 13.1 Fiber 层：顶层 effects 并行

`\_disposables.clear()` 先拿到当前所有顶层 wrappers，然后 `Promise.all` 并行回收。每个 wrapper 都由 composeError 包裹；单个 cleanup 失败只记录，不阻断其它 effect 的释放。

## 13.2 Effect 层：局部 disposers 逆序串行

每个 wrapper 内部的 `dispose()` 对自己的局部 disposer 做 reverse；遇到异步后把后续 cleanup 串到 Promise 上。因此：顶层 effect 之间可以并行，单 effect 内部保持 LIFO。

## 13.3 为什么这不是严格事务 rollback

cleanup 可能失败，而且顶层 effects 并行会改变跨 effect 的全局顺序。所以 Cordis 的工程语义更像“结构化、可等待、尽最大努力的可逆资源管理”，而不是数据库式全局 ACID rollback。设计上假设互相有顺序依赖的资源应放进同一 effect ownership 层级或显式编排。

## 13.4 unload 尾部就是 reconciliation

清完后 `store=undefined`。若最新 epoch 仍 INACTIVE，lifecycle 稳定，inertia 清空；若清理期间依赖又恢复，则直接 `\_reload()` 并进入 LOADING。这使系统不会因为“正在卸载时服务回来了”而错过恢复。

# 14. `inertia`：隐藏的 lifecycle mutex

`inertia` 不是“惰性”的业务概念，而是当前生命周期过渡 Promise。其三个职责：

- 防止 `\_setEpoch()` 在已有 load/unload 时再开启并发过渡；只更新目标 epoch。
- `\_dispose()` / `await()` 能等待链式 transition 完全结束。

- reload 尾部接 unload、unload 尾部接 reload 时，inertia 被替换为下一段 Promise，因此 `await()` 必须 while 循环，而不是只 await 一次。

伪代码：

```
while (fiber.inertia):  
    await fiber.inertia  
if fiber._error: throw fiber._error
```

## 15. 状态函数：为什么 state 不能直接等于 epoch

### 15.1 `\_getState()`：稳定状态派生

优先级是：uid==null → DISPOSED；有 `\_error` → FAILED；runner epoch 非 INACTIVE → ACTIVE；否则 PENDING。注意 LOADING/UNLOADING 不由 `\_getState()` 推导，而是 `\_updateState(callback)` 在过渡开始时显式返回。

### 15.2 `\_updateState()` 为什么只在 ACTIVE 边界通知 service

Fiber 自己提供的 services 只有在其 Fiber ACTIVE 时才应该对依赖者可见。状态跨越 ACTIVE↔non-ACTIVE 时，遍历 Reflect store 中由该 Fiber 提供的 Impl 并 notify；PENDING↔FAILED 这类都属于 non-ACTIVE，不需要重复广播 service availability。

### 15.3 `internal/status` 先发，service notify 后发

观察者先看到 Fiber 状态改变，再由 service dependency graph 响应。这允许监控/loader 获取清晰的 transition event，同时下游依赖刷新仍由 ReflectService 统一触发。

## 16. 末端 API 逐函数拆解

### 16.1 `name` getter

从当前 Fiber 向 parent Fiber 走，返回最近有 runtime.name 的祖先；root 返回 `root`。这意味着匿名 wrapper plugin 的日志可以继承更有意义的父插件名。

### 16.2 `assertActive()`

这里只阻止 `uid === null` 的永久 disposed Fiber。PENDING/FAILED 仍可执行一些结构性操作；真正禁止新 effect 在卸载期发生的是 `effect()` 内的额外 `state===UNLOADING` 检查。

### 16.3 `getEffects()`

读取 Fiber 当前 `\_disposables`，提取绑定在 wrapper 上的 EffectMeta，形成可诊断的 effect ownership forest。可以把它作为泄漏排查和 runtime inspector 的基础接口。

### 16.4 `await()`

循环等待 inertia 直到稳定，然后若 `\_error` 存在就 rethrow。它的语义不是“等待一次 load”，而是“等待当前所有由依赖变化诱发的 load/unload 链收敛”。

## 16.5 `\_restart()`

先 `\_setEpoch(INACTIVE)` 强制当前 activation 失效，再 `\_refresh()` 立即根据现有依赖算回 active epoch。如果依赖仍满足，过程会自然变成 ACTIVE→UNLOADING→LOADING→ACTIVE，而不是直接重复 callback。这样旧 effects 必须先回收。

## 16.6 `\_update(config)`

先保存 raw `\_config`。若当前不是 ACTIVE，就清错误并通过 setEpoch/refresh 触发未来合适时机激活，不立即 config resolve；若 ACTIVE，则先 `\_resolveConfig`，再进入 `internal/update` waterfall，默认行为是更新 `config`、清 `\_error`、restart。HMR/持久化层可以在 waterfall 中替换默认策略。

# 17. 五条主线合并成一条真实执行链

10. Service notify 使 `\_checkImpl` 更新 `\_store`。
11. `\_refresh` 把 Provider UIIDs 编成新 epoch。
12. `\_setEpoch` 记录目标；若当前没有 inertia，决定启动 `\_reload` 或 `\_unload`。
13. `\_reload` 快照 store/epoch，经过 microtask stale check，解析 config，执行 `\_runner`。
14. `\_runner` 的 `\_execute` 收集插件启动产生的顶层 Effect wrappers。
15. 每个 `effect()` 又拥有自己的局部 disposer 栈，并在创建时先注册 wrapper 以抵抗 reentrancy。
16. 任何 reentrant dispose 都通过 setupBarrier/inFlight/effectInertia join 正确的 teardown。
17. `\_unload` 并行回收顶层 effects；每个 effect 内部逆序清理。
18. load/unload 尾部重新观察最新 epoch，直到最终 `inertia=undefined`。

# 18. 六个具体时序案例

## 案例 1：正常启动 - database 已存在

```
PENDING
_checkImpl(database) -> Provider #12
_refresh() -> epoch=:12
_setEpoch(:12) -> LOADING
_reload()
  store={database:#12}
  config resolve
  _execute(_runner)
-> ACTIVE
```

## 案例 2：Provider #12 被 #37 替换

```
ACTIVE epoch=:12
notify(database)
_store.database=#37
_refresh -> :37
_setEpoch -> UNLOADING
_unload old effects
latest epoch still :37
_reload -> ACTIVE epoch=:37
```

### 案例 3：load 刚排上微任务，依赖立刻消失

```
_setEpoch(:12) -> _reload scheduled
service removed -> _runner.epoch=INACTIVE (inertia exists, no second transition)
_reload microtask resumes
oldEpoch=:12 != current INACTIVE
=> skip plugin callback
=> tail enters _unload / settle PENDING
```

### 案例 4：effect setup 内同步 dispose parent

```
effect()
  wrapper push owner._disposables # 先可见
  _execute(setup)
    setup -> parent.dispose()
      owner _unload captures wrapper
      wrapper sees executing=true
      waits setupBarrier
    setup returns disposer
  barrier resolves
  wrapper disposeAfter -> cleanup
=> 无孤儿 effect
```

### 案例 5：用户先调用 disposer，外层随后 unload

```
public wrapper() -> async cleanup inFlight=P
second direct wrapper() -> no-op
owner unload -> runDisposable(wrapper)
  direct call no-op
  WeakMap returns inFlight=P
owner awaits P
=> 所有权语义仍完整
```

### 案例 6：PENDING child 在 publication observer 中注册 effect 后立即 dispose

```
child created, epoch=INACTIVE
internal/plugin observer -> child.ctx.effect(...)
observer/parent immediately dispose child
_setEpoch(INACTIVE) sees no epoch transition
DeepSeek fallback: if !inertia -> force _unload()
=> pre-activation effect still cleaned
```

## 19. 不变量 (Invariants)：读源码时最值得记住的 12 条

19. uid 为 null 后 Fiber 永远不应创建新的 effect。
20. UNLOADING 时不允许注册新 effect，避免越过 unload snapshot。
21. 只要 effect teardown 未 settle，其 wrapper 对 owner 必须仍可见或可通过 inertia join。
22. 同一个 effect 的实际 cleanup 最多启动一次；其它调用者 join 同一结果。
23. Fiber 同时最多只有一条 lifecycle transition 链（`inertia`）。
24. epoch 更新不会因为已有 inertia 而被丢弃，只会延迟处理。
25. 插件 callback 只能在 reload 的 oldEpoch 仍然有效时运行。
26. 当前 activation 的 service 视图来自 `store` 快照，而 dependency detection 写 `\_store`。

27. Provider identity 改变等同于 Consumer activation 失效。
28. Fiber ACTIVE 边界变化必须 notify 其提供的 services。
29. 结构性 child ownership 必须先于 publication 建立。
30. 单个 observer/cleanup 失败不应阻断其它结构性 cleanup；错误通过 logger 保留。

## 20. Gotchas：读源码或二次开发最容易踩的坑

| Gotcha                                      | 后果                            | 建议                                       |
|---------------------------------------------|-------------------------------|------------------------------------------|
| 把 epoch 当 bool                              | 看不懂 Provider 替换为何 reload      | 把它理解成 dependency-version vector 的字符串编码   |
| 只看 `state` 不看 `inertia`                     | 误判 lifecycle 已稳定              | 诊断时同时输出 state、epoch、inertia              |
| 在 UNLOADING callback 里新建 effect             | DeepSeek 会直接报 INACTIVE_EFFECT | 把资源创建放在下一次 activation                    |
| 假设 Fiber cleanup 全局 LIFO                    | 跨顶层 effect 有并行 teardown 竞态    | 有顺序依赖的资源放在同一 effect 内                    |
| config hook 访问 injected service 但提前 resolve | PENDING/FAILED 时可能无 service   | 沿用 `_config` raw + activation resolve 模式 |
| 自定义 disposer 第二次调用返回空，就认为 owner 不需等待        | 可能仍有第一次异步 cleanup             | 结构 owner 必须使用可 join 机制                   |
| 在 publication observer 中做不可逆外部操作            | reentrant rollback 只能回收受控资源   | observer 应尽量只注册可追踪 effect                |
| 修改 `_store` 却不 `_refresh`                   | epoch 不更新                     | service 变化入口统一走 Reflect.notify           |

## 21. 如何测试 `fiber.ts`：建议的竞态测试矩阵

| 测试                                      | 触发方式                                   | 必须断言                        |
|-----------------------------------------|----------------------------------------|-----------------------------|
| 同步 setup + 正常 dispose                   | effect 返回 disposer                     | cleanup 一次；二次 dispose no-op |
| async setup + dispose before resolve    | Promise 未完成先 dispose                   | cleanup 等 setup 完成后执行       |
| sync setup throw after partial collect  | generator/嵌套 effect 后 throw            | 已收集资源回滚；错误原样抛出              |
| public dispose + owner unload overlap   | 先 wrapper，再 parent dispose             | owner 等待同一 inFlight cleanup |
| effect setup reentrant parent dispose   | setup 内触发 parent.dispose               | 不泄漏；无重复 cleanup             |
| plugin publication reentrant dispose    | internal/plugin listener dispose child | constructor 不继续激活           |
| PENDING observer effect + child dispose | publication listener 注册 effect         | 强制 unload 清掉 effect         |
| reload stale epoch                      | _reload checkpoint 前 remove provider   | plugin callback 不应执行        |
| provider identity swap                  | A→B 同名 service                         | consumer 完整 unload/reload   |
| rapid A→B→A                             | inertia 期间连续 epoch 改变                  | 不并发 transition；最终收敛最新 epoch |
| cleanup throw                           | 一个 disposer 抛异常                        | 其它顶层 effect 仍被回收            |
| config update while PENDING             | 缺依赖时 update raw config                 | 不提前 resolve；激活时使用最新 config  |

## 22. 与论文 “Revertible Effects + Reactive Coeffects” 的一一映射

| 论文概念                   | fiber.ts 落点                                                        | 工程化修正                                   |
|------------------------|--------------------------------------------------------------------|-----------------------------------------|
| Revertible Effect      | <code>`effect()`</code> 收集 disposer, <code>`_unload()`</code> 触发恢复 | 不是数学严格 inverse; 允许 cleanup 失败并记录        |
| Effect Context         | Fiber <code>`_disposables`</code> + child ownership                | effect metadata 构造 ownership tree       |
| Reactive Coeffect      | <code>`inject`</code> + <code>`_checkImpl/_refresh`</code>         | provider UID epoch 追踪身份变化               |
| Temporal composability | restart/unload/cleanup                                             | reentrant teardown join、PENDING drain   |
| Spatial composability  | service Provider→Consumer lifecycle                                | store snapshot + active boundary notify |
| Component lifecycle    | FiberState + inertia                                               | 单 in-flight reconciliation              |

论文的形式化表达强调“effect 有 inverse”；实际源码更现实：disposer 是恢复程序，可能 async、可能失败、可能部分完成。DeepSeek 的主要工作并非改变论文模型，而是把“恢复程序本身也处在复杂异步生命周期里”这一工程事实纳入 runtime。

## 23. 设计评价：为什么这 709 行值得学习

### 23.1 优点

- 把插件 callback、局部 effect、child plugin 都统一成 ownership + disposer 模型，语义压缩很强。
- epoch 用 Provider identity 而非布尔 availability，能可靠触发 capability replacement 的完整重绑定。
- inertia 实现了轻量的 reconciliation serializer，避免 load/unload 并发互撕。
- DeepSeek 的 reentrant hardening 非常贴近 JavaScript 插件系统真实故障模式，尤其是“同步 callback 里反向 dispose”。
- 错误隔离策略清晰：结构 cleanup 尽量继续，错误记录；startup error 则通过 ``await()`` 重新暴露给调用者。

### 23.2 代价与复杂性

- ``effect()`` 已经变成一个小型异步状态机，维护成本高；变量间不变量需要测试而不只是注释。
- 顶层并行 cleanup 对跨 effect 隐式顺序依赖不友好，框架使用者必须正确建模 ownership。
- epoch 字符串依赖 inject key 顺序与 UID，适合 identity reconciliation，但不是通用版本向量。
- 通过 PromiseLike disposer 统一 sync/async API 很巧，但对新读者直觉不强。
- observer 隔离与 logger failure 的处理仍存在“日志系统本身失败即无法恢复”的底层边界。

## 24. 如果你要迁移/复刻这套机制，最小实现顺序

31. 先实现 Fiber + owner disposer list，不做 DI。
32. 实现 ``effect(setup)->disposer``，确保 local LIFO、single-shot、异常回滚。
33. 把 child plugin 也注册成 parent effect，形成 ownership tree。
34. 加入 service registry + inject，区分 ``_store`` 实时候选与 ``store`` activation snapshot。
35. 引入 provider identity epoch，先只处理 active/inactive。
36. 加入 ``inertia``，让 epoch 快速变化只保留单 transition 链。
37. 最后加入 reentrant hardening：wrapper-before-execute、setupBarrier、inFlight join、pending effect drain。
38. 再做 config waterfall/HMR；不要一开始就把 loader、schema、event 全揉在一起。

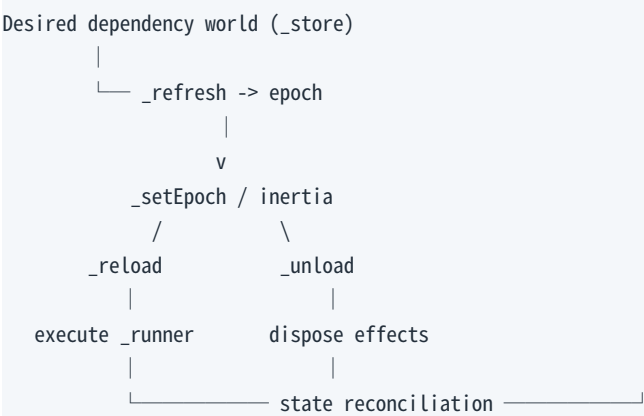
## 25. 建议的源码阅读断点

| 断点                  | 观察变量                         | 你应该看到什么                      |
|---------------------|------------------------------|------------------------------|
| constructor L229 附近 | uid/state/_runner.epoch      | 普通 child 初始 INACTIVE/PENDING |
| effect L393 入口      | state/_disposables           | wrapper 在 execute 前进入 owner  |
| _execute L332       | oldEpoch/runner.epoch        | async effect 世代快照            |
| _refresh L572       | _store/inject/epoch          | 依赖 UID 被编码                   |
| _setEpoch L585      | old/new epoch/inertia        | 已有 inertia 时只改目标             |
| _reload L604        | store/oldEpoch/current epoch | microtask 后 stale check      |
| _unload L632        | cleared wrappers/inFlight    | 顶层并行、嵌套 join                 |
| update L690         | _config/config/state         | PENDING 时 config 延迟解析        |

## 26. 一页式心智模型

把 Fiber 想成一个“可重放、可撤销、依赖感知的 activation transaction”。`\_runner` 描述如何启动；epoch 描述这次启动依赖哪个世界；effect 记录如何撤销；inertia 保证同一时刻只有一个生命周期事务在推进；reload/unload 则不断对齐“当前 activation”与“最新 desired epoch”。

心智模型：



## 附录 A. Source line map（DeepSeek Harness 709 行版本）

以下行号以 2026-08-19 检索到的 `deepseek-ai/deepseek-harness/master/vendor/cordis/src/fiber.ts` 为准；仓库快速迭代后可能漂移。

| 行号      | 重点                                               |
|---------|--------------------------------------------------|
| 95-108  | EffectRunner、effectInertia、runDisposable         |
| 109-127 | emitPluginDisposed observer 隔离                   |
| 170-312 | Fiber 字段与 constructor publication/ownership      |
| 229-245 | 插件级 `_runner`                                    |
| 246-278 | child structural disposer + PENDING effect drain |
| 279-298 | publication reentrancy 防线                        |



|         |                                                  |
|---------|--------------------------------------------------|
|         |                                                  |
| 332-376 | `_execute` 多形态 Effect 执行器                        |
| 393-527 | `effect()` 完整实现                                  |
| 473-488 | wrapper + effectInertia + wrapper-before-execute |
| 489-515 | setup failure/async rejection rollback           |
| 539-558 | 状态派生与 ACTIVE 边界服务通知                              |
| 559-599 | provider check、epoch、transition serializer       |
| 600-631 | config resolution + stale-safe reload            |
| 632-653 | unload + reconciliation                          |
| 660-706 | await/restart/update                             |

## 附录 B. 参考资料与版本说明

1. DeepSeek Harness vendored Cordis fiber.ts (709 lines, retrieved 2026-08-19):  
<https://github.com/deepseek-ai/deepseek-harness/blob/master/vendor/cordis/src/fiber.ts>

2. Upstream Cordis core fiber.ts (about 458 lines at retrieval):  
<https://github.com/cordiverse/cordis/blob/main/packages/core/src/fiber.ts>

3. Cordis paper repository, “A Programming Paradigm for Spatiotemporal Composability” : <https://github.com/cordiverse/paper>

4. Cordis core repository: <https://github.com/cordiverse/cordis>

5. DeepSeek Harness repository: <https://github.com/deepseek-ai/deepseek-harness>

分析方法：以源码行为为主，以论文的 Reversible Effects / Reactive Coeffects 作为语义解释框架；不把 disposer 等同于数学上的严格逆元。文中代码块均为为解释而写的伪代码/简化结构，而非大段复制源码。